Authored by: Arpit Raj, LNMIIT Jaipur
Date: October 24, 2026
© 2026 Arpit Raj. All rights reserved.

Inheritance in OOP: Access Modifiers & Types

Inheritance Access Matrix

Table: Base Member vs Type of Inheritance

Base member (↓) / Type of Inheritance (→)	public	protected	private
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

- If you see last row all **public**, **protected** and **private** types cant access **private** members of parent class.
- So if a child needs to interact with private variable from parent class, it can do so using public / protected getter / setter functions.

Eg

```
class Parent {
public:
    int x=10;
protected:
    int y=10;
};

class child : private Parent {
public:
    void show() {
        cout << x << " " << y;
    }
};

main() {
    child obj;
    obj.show(); // -> OK
    cout << obj.x; // -> ERROR
}
```

in simple words, Private inheritance mode inherited member private within child, so it's inaccessible by outside world or derived classes.

> **Friend Classes/Functions Rules**
> - a friend of a derived class does not automatically make it a friend of base class
> - a derived class itself is not automatically a friend of its base class

The 'IS A' Relationship

Inheritance is used whenever there is a hierarchical 'IS A' relationship identified between objects.

Classic example of inheritance:

```
Base class (generalized class): Vehicles (name, model, no of tyres, start(), stop())
      |
      |———> Branch 1 -> Child class: Car (no of door, chases, start AC())
      |
      └———> Branch 2 -> Child class: Bike (handlebar, wheelie())
```

- **Note:** Child class inherits base property plus their unique property (specialized class).

- inheritance helps to follow DRY principle (DO NOT repeat yourself) i.e promotes reusability.

> **REUSABILTY**
> [avoids duplicating methods in child class that already exist]

- in future if a new class comes which is a specialized class of our general class, we can simply make it inherit common properties.

> **EXTENSABILTY**
> [easy upgrades of specific parts of product without changing core attributes]

Sign convention:
+ public
- private
protected

Why use Protected?

Why we required protected access modifier
Eg consider a case where we want child class to inherit a property but we dont want main() or any external code to access that property - then if we didnt have protected we had 2 choices

- case 1: `+x` (public) -> then child will inherit `x`, but main can also access it
- case 2: `-x` (private) -> even child cant access `x` as private members are not inherited

© Arpit Raj, LNMIIT Jaipur

Hence we use protected because it allows child class to inherit but acts private for outside external code.

hence `P #xyz` branches to `c1` and `c2`.

- both `c1`, `c2` gets a copy of `xyz` which is also protected inside `c1`, `c2`
- so `P.xyz` and `C.xyz` both wont run

2 primary purpose of protected

1. **Encapsulation:** promotes data hiding and prevents external code from modifying or accessing sens data directly
2. **Inheritance:** derived class have access to these member allowing them to build upon base class while maintaining some level of data integrity and control

Mode of inheritance is specific to C++ public, private, protected mode of inheritance.

- in Java there is only public inheritance

© Arpit Raj, LNMIIT Jaipur

Types of Inheritance

1. **Single inheritance:** single subclass inherits from only one superclass.

```
Diagram: P1 points up to from C1
  P1
  ↑
  C1  (C1 -> P1)
```

   - supported in all C++, python, Java

2. **Multiple inheritance:** single subclass inherit from more than one superclass.

```
Diagram: P1, P2 point down to C1
  P1     P2
    ↘     ↙
      C1
```

   - supported in C++, python
   - Java uses interface to handle it, multiple inheritance isnt for Java classes

3. **Multilevel inheritance:** class derived from class which is already derived.

```
Diagram:
  P1
  ↓
  C1
  ↓
  C2
```

   - C2 is multilevel inheritance

© Arpit Raj, LNMIIT Jaipur

4. **Hierarchical inheritance:** multiple subclasses from a single superclass.

```
Diagram: P points down to C1 and C2
       P
      / \
     /   \
    ↓     ↓
   C1     C2
```

5. **Hybrid inheritance:** a mix of one or more type of inheritance.
   - once again not in Java classes, supported in C++, python
   - Java can achieve it using interfaces

```
Diagram: P points down to C1 and C2, both C1 and C2 point down to C3 (diamond stru
       P
      / \
     /   \
    ↓     ↓
   C1     C2
    \     /
     \   /
      ↘ ↙
       C3
```